10 · 系统设计与场景题

技术面试准备材料 · AI 应用 / Agent 开发

> 系统设计题不要一上来画架构图。先问清楚目标、规模、约束和失败代价，再给方案和取舍。

1. 设计一个测试全流程多 Agent 平台

先明确目标：把测试流程里可标准化的环节交给 Agent，人负责判断和兜底。范围包括需求分析、测试设计、用例生成、环境准备、执行和结果分析。

容量与约束：测试任务通常不是超高频，但单个任务可能很长、工具调用多、需要保留完整轨迹。核心约束是可靠性和可追溯，而不是极致 QPS。

架构分四层：
1. 接入层：任务 API、Webhook、CLI；同步请求只创建 Job，长任务异步执行。
2. 编排层：Agent Pool + 固定/模板化 Workflow；根据输入资产决定需要哪些阶段；DAG 校验依赖和数据流。
3. 执行层：Agent 运行时管理状态、上下文、工具调用、预算和检查点；工具层做权限、沙箱、幂等。
4. 支撑层：模型网关、知识检索、记忆、评测、可观测性、审计。

数据流：需求文档 → 测试点 → 用例 → 执行计划 → 执行结果 → 缺陷/报告。每一步有输入输出契约，结果落库并保留轨迹。

失败处理：工具幂等 + 检查点 + 租约；可重试的退避重试，不可重试的转人工；高风险动作（改环境、提缺陷）审批。

取舍：生产优先固定/模板化 Workflow，动态生成只在没有模板时兜底；Agent 按能力边界拆，不按角色数量拆；核心链路自己掌控，外围能力复用。

衡量：任务成功率、测试点覆盖率、人工修改率、单任务时延和成本、使用率。

2. 设计一个企业知识库问答 Agent

目标：让员工用自然语言问业务问题，答案有依据、有权限、可追溯。

约束：文档多、更新频繁、权限复杂、答案错误代价高。所以优先保证「不胡编」和「不越权」，而不是回答率。

架构：
1. 接入与身份：SSO，拿到用户、部门、角色。
2. 文档处理：解析 PDF/HTML/表格/图片 → 清洗 → 结构化切分 → 元数据（来源、密级、部门、版本）。
3. 索引：向量 + 关键词混合索引；权限字段写入索引。
4. 检索：查询改写 → 多路召回 → 权限过滤 → Rerank → 上下文组装。
5. 生成：基于资料回答，强制引用；资料不足时拒答。
6. 评测与反馈：标注集、引用准确率、拒答准确率、人工抽检。

关键设计：
- 权限在检索阶段过滤，不能生成后再过滤；

- 多租户/部门隔离贯穿缓存和日志；
- 文档更新用版本 + TTL，删除要同时删向量和缓存；
- 外部文档视为不可信数据，防间接 Prompt Injection。

取舍：RAG 优先，长上下文只用于少量需要整体理解的任务；不微调灌知识；回答率和准确率冲突时选准确率。

衡量：答案正确率、引用准确率、拒答准确率、检索延迟、用户采纳率。

3. 设计一个高并发 Agent 网关

目标：在多地多租户场景下，稳定地把请求路由到合适的模型和 Agent，过载时可预测地降级。

容量：先用 Little's Law 估并发，按 P95/P99 而非平均值规划；利用率留 20% 余量；区分交互请求和后台任务。

架构：
1. 接入层：鉴权、租户识别、请求规范化。
2. 准入层：多级限流（用户/租户/接口/全局）、基于预计完成时间的准入控制、有界队列。
3. 路由层：模型路由（能力/成本/延迟/健康度）、Agent 路由、灰度与 A/B。
4. 执行层：连接池、超时、重试、熔断、跨模型 fallback、流式转发。
5. 缓存层：精确缓存、语义缓存、工具结果缓存，按租户隔离。
6. 观测层：TTFT、P95/P99、错误率、429、Token、成本、降级次数。

关键设计：
- 重试只在一层做，避免叠加；
- 限流键要选对，防止单租户打满；
- 语义缓存要防假阳性和跨租户泄露；
- 过载时优先保交互请求，后台任务排队或拒绝；
- 所有降级可配置、可灰度、可回滚。

取舍：一致性和可用性之间，优先保证不雪崩；复杂路由增加延迟，需要缓存决策结果；本地限流快但精度低，Redis 准但有延迟，通常混合。

4. 设计一个排障 Agent

目标：把专家的排障路径变成可复用、可验证、可控制的流程，而不是让模型自由发挥。

约束：故障时信息不全、时间紧、恢复动作有风险。核心是证据驱动和受控恢复。

架构：
1. 故障接入：告警、日志、指标、变更、执行轨迹，统一成故障上下文。
2. 分类：按症状、影响面、组件分层分类，先缩小范围。
3. 假设生成：多个根因假设，每个假设对应可验证的检查动作；按信息增益选择下一步。
4. 诊断 Workflow：动态生成但经过校验；工具只读优先；每步记录证据。
5. 恢复：动作按风险分级；低风险自动，高风险审批；执行前验证门，执行后回滚方案。
6. 经验库：结构化保存现象、根因、方案、验证结果和适用边界。

关键设计：
- 结论不是一个布尔值，而是「根因 + 置信度 + 证据 + 建议」；

- 证据不足时明确说不足，不硬下结论；
- 恢复动作和诊断动作分开权限；
- 所有动作可审计、可回放。

取舍：动态诊断灵活但不可复现，固定脚本稳定但覆盖窄；生产上用「固定分类 + 动态检查 + 受控恢复」的混合模式。

衡量：根因命中率、平均定位时间、误判率、自动恢复成功率、人工接管率。

5. 设计一个风控 Agent

目标：增强传统风控的调查和解释能力，而不是替代在线决策。

边界：
- 在线实时决策仍由规则/评分卡/传统模型负责，要求毫秒级、可解释、可审计；
- Agent 负责离线或准实时调查、证据汇总、可疑点解释、未知风险发现；
- Agent 只能产出建议或候选，不能直接放行、冻结或改规则。

架构：
1. 在线链路：交易/行为 → 规则与模型评分 → 决策（通过/拦截/人工）→ 记录。
2. 调查链路：命中或人工触发 → 收集证据（历史行为、关联账户、设备、关系网络）→ Agent 生成假设 → 验证 → 调查报告。
3. 工具网关：六道关（身份、权限、参数、频率、审批、审计）；权限集里「没有什么」比「有什么」更重要。
4. 结论门：确定性规则校验 Agent 结论，只能输出候选，不能直接生效。
5. 反馈：人工结论回流，更新规则、评测集和经验库。

关键设计：
- 高风险动作动作级审批，不是权限级；
- 改规则必须走审批和灰度；
- Agent 的「未知风险发现」只能到 CANDIDATE，不能自动上线；
- 所有证据和结论可追溯。

衡量：调查效率、人工采纳率、误报减少、未知风险发现数、合规审计通过率。

6. 设计一个长任务异步执行系统

目标：让可能运行几分钟到几小时的任务可靠完成，支持查询、取消、恢复。

架构：
1. API：创建 Job 返回 ID；查询状态；取消；订阅事件。
2. 状态机：PENDING → QUEUED → RUNNING → WAITING_APPROVAL / SUCCEEDED / FAILED / CANCELLED。
3. 队列：有界、优先级、按租户公平调度；长任务和交互任务分开。
4. Worker：拉取任务、续租、执行、心跳、写检查点。
5. 存储：Job 表、检查点表、事件表；状态和事件都保存，便于恢复和审计。
6. 恢复：租约过期由 Reaper 回收；恢复时读检查点并核对工具状态，决定重放/跳过/转人工。
7. 幂等：工具调用带幂等键；消息消费幂等；Outbox 保证状态与消息一致。

关键设计：
- 检查点粒度在关键步骤和工具调用前后；

- 取消要传到上游模型和工具，并保存状态；
- 重试有全局预算，失败进 DLQ；
- 长任务不能占用同步连接。

衡量：任务完成率、恢复成功率、重复副作用率、P95 完成时间、队列等待时间。

7. 设计一个多租户 Agent 平台

目标：多个业务方共享平台能力，但数据、权限、配额、模型配置互相隔离。

架构：
1. 租户层：租户、用户、角色、配额、计费。
2. 配置层：每个租户独立的 Prompt、模型、工具、知识库、Workflow 版本。
3. 执行层：共享 Agent 运行时和工具网关，但按租户注入权限和配额。
4. 数据层：租户 ID 贯穿存储、索引、缓存、日志、队列；敏感数据可物理隔离。
5. 观测层：按租户下钻的指标、审计和账单。

关键设计：
- 默认隔离、显式共享；跨租户访问必须授权；
- 缓存 key 带租户和权限；
- 限流和队列按租户公平调度，防止大租户饿死小租户；
- 配置版本按租户管理，支持灰度和回滚；
- 计费按模型调用、工具调用、存储分别统计。

取舍：共享平台降低运维成本，但隔离和噪声邻居问题更复杂；高价值租户可以提供独占资源，但不能破坏整体公平性。

衡量：租户隔离测试通过率、单租户故障影响面、资源利用率、配置发布效率。

8. 自研还是用现成框架？怎么选？

我会按「核心差异」和「可控性要求」判断。

如果某个能力是业务核心竞争力，或者现成框架在可观测性、失败恢复、权限上不满足要求，我会自研核心链路。比如工具执行的幂等、检查点恢复、网关的重试和降级，这些直接决定生产可靠性，最好自己掌控。

如果是通用能力，比如模型 SDK、向量库、Web 框架、消息队列，优先复用，不值得重复造。

框架的问题通常不在「能不能跑」，而在「出问题能不能查、能不能改」。选框架时要看：失败处理是否可插拔、是否支持自定义存储、是否有追踪、社区是否活跃、升级是否可控。

我的策略是「核心自研、外围复用、接口隔离」：核心逻辑自己写，外围通过适配器接入，方便替换。

不要为了简历造轮子，也不要为了省事把关键链路交给不透明的框架。

9. 新能力怎么做上线和灰度?

分四步：离线验证、小流量灰度、逐步放量、全量。

离线：跑评测集，看核心指标和回归；对高风险场景做安全测试。

灰度：按用户、租户、流量比例或场景灰度；新老版本并行，保留一键回滚；灰度期间重点看错误率、延迟、成本、人工修改率，而不只是成功率。

放量：设定明确的放量条件，比如连续 `N` 天指标达标、无 `P1` 问题；每次放量后观察一段时间。

全量后：继续监控，保留回滚能力；把线上失败样本补进评测集。

灰度策略要避免「只灰度简单流量」，否则复杂场景的问题发现不了。分层抽样，确保覆盖典型和边界场景。

另外，`Prompt`、模型、工具、知识库任何一项变化都应该走灰度，不能只把代码发布当成上线。

10. 如何做 Agent 成本优化?

先建立成本基线：单任务平均 `Token`、模型单价、工具调用次数、重试和失败成本。没有基线就无法优化。

优化手段分四类：
1. 减少调用：缓存、`Singleflight`、批处理、避免重复检索。
2. 降低单价：模型分级路由，简单任务用小模型；长 `Prompt` 用缓存；能本地跑的用本地。
3. 减少 `Token`：上下文压缩、工具结果摘要、结构化状态替代长历史、检索只取相关片段。
4. 减少浪费：超时和取消、失败快速退出、限制最大步数、避免无效重试。

但要算总账：便宜模型可能失败率更高，重试和人工成本反而更大；过度压缩可能降低效果，增加人工修改。

我的做法是按场景设预算和 `SLA`，在预算内选效果最好的方案；超过预算就降级或转人工，而不是无限制调用。

成本优化不能牺牲可靠性，尤其是高风险场景。

11. 如果模型能力提升 10 倍，你的架构会怎么变?

不会把架构推翻，但会调整重心。

第一，编排会更轻。模型能处理更长的上下文和更复杂的计划，很多为弥补能力不足而做的拆分会减少；但确定性校验、权限和审计不会消失。

第二，上下文工程更重要。上下文变长不代表可以乱塞，选择、压缩和隔离仍然决定效果和成本。

第三，评测和归因更关键。模型越强，错误越隐蔽，越需要可验证的证据和轨迹。

第四，安全边界不变。模型再强也可能被注入、会犯错，工具权限、沙箱、审批仍然必要。

第五，成本结构变化。如果推理更便宜，一些缓存和分级策略可以简化；但如果上下文更长，`Token` 总量可能反而上升。

> 我的判断是：模型能力提升会减少「补丁式工程」，但不会替代可靠性、评测、安全和可观测性。这些是系统的底盘，不随模型版本变化。

12. 面试官问「你还有什么技术判断想分享」，怎么答？

> 我会讲一个我真正相信的判断：`Agent` 的竞争力不在模型调用，而在「约束下的可靠性」。
>
> 模型能力是公共资源，谁都能用；但能不能把工具安全地执行、把失败恢复、把效果评测、把成本控制住，是每个团队自己的工程能力。工业场景里，用户不会因为「用了大模型」就买单，他们只会因为「事情办成了、没出错」而持续使用。
>
> 所以我在项目里花最多时间的不是调 `Prompt`，而是幂等、状态恢复、评测集、权限和可观测性。这些东西不显眼，但决定了系统能不能从 `demo` 走到生产。
>
> 另一个判断是「不要过度 `Agent` 化」。能用固定流程解决的就用流程，`Agent` 只放在真正需要判断的地方。自主性是有成本的，边界清楚比能力强大更重要。
>
> 如果面试官继续追问，我会用一个具体例子说明：某个工具调用崩溃后如何避免重复副作用，以及这个设计如何影响整体架构。

高频追问

- 系统设计题里，你先问哪三个问题？
- 如果 QPS 只有 10，但每个任务跑 10 分钟，架构要怎么变？
- 多租户平台如何做噪声邻居隔离？
- 风控 Agent 的结论如何进入人工流程而不影响时效？
- 模型降价后，哪些缓存策略应该取消？
- 你设计的系统里，最可能先出问题的是哪一环？